

TRƯỜNG ĐẠI HỌC VINUNI
CHƯƠNG TRÌNH AI THỰC CHIẾN 2026 - KHÓA 2

PHẠM THỊ THẨM
Mã số học viên: 2A202600789

BÁO CÁO CHẠY THỜI GIAN THỰC
DỰ ĐOÁN GIAN LẶN TRONG GIAO DỊCH

Chương 1

TỔNG QUAN HỆ THỐNG

1.1 Mục Tiêu

Hệ thống mô phỏng một đường ống phát hiện gian lận giao dịch tài chính theo thời gian thực với các chức năng chính:

- Nhận dòng giao dịch "mới" đến liên tục thông qua Kafka (broker trung gian).
- Tính toán đặc trưng (features) và lịch sử giao dịch của từng khách hàng theo thời gian thực.
- Sử dụng mô hình học sâu LSTM + Attention để dự đoán xác suất gian lận của từng giao dịch.
- Lưu toàn bộ kết quả dự đoán vào PostgreSQL phục vụ đánh giá độ chính xác và đo độ trễ (latency).

Hệ thống dùng tập dữ liệu giao dịch mô phỏng (được xây dựng dựa trên lý thuyết, phân phối và quy luật, code giống với tập dữ liệu dùng để train mô

1.2. CÔNG NGHỆ SỬ DỤNG

hình) lịch sử từ 2018-10-01 đến 2020-05-22 lưu dưới dạng các file .pkl ngày. Và "phát lại" (replay) chúng với tốc độ 50 giao dịch/giây, 100 giao dịch/giây, 500 giao dịch/giây để mô phỏng dữ liệu đến theo thời gian thực.

1.2 Công Nghệ Sử Dụng

Bảng 1.1: Công nghệ sử dụng trong hệ thống

Thành phần	Công nghệ / Phiên bản
Message broker	Apache Kafka 4.3.1 (chế độ KRaft, 1 node)
Xử lý luồng dữ liệu	Apache Spark 4.0.1 – Structured Streaming
Mô hình dự đoán	PyTorch – LSTM + Attention (chạy trên CPU)
Cơ sở dữ liệu	PostgreSQL 16
Điều phối hạ tầng	Docker Compose (docker-compose.yml)
Producer / giám sát	Python (kafka-python, pandas, psycopg2, Docker CLI)

1.3 Dữ Liệu Đầu Vào

Tập dữ liệu gồm 600 file .pkl, mỗi file chứa toàn bộ giao dịch của một ngày (định dạng tên yyyy-mm-dd.pkl). Mỗi giao dịch gồm 8 cột:

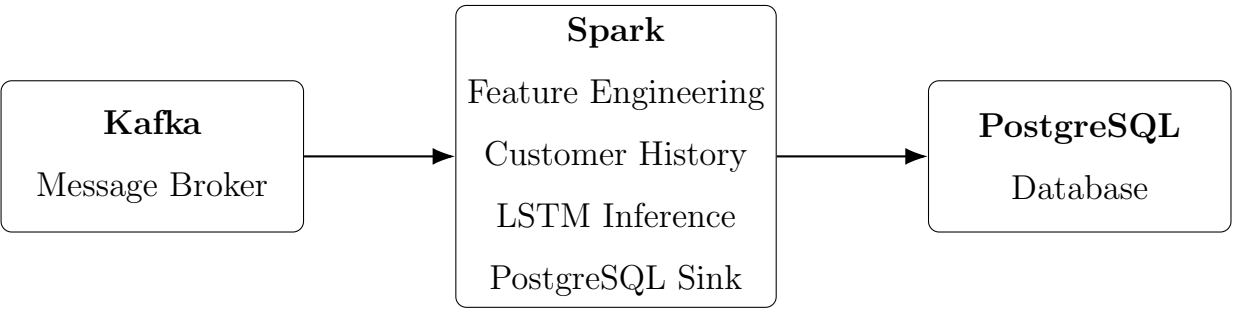
1.4. SƠ ĐỒ TỔNG THỂ PIPELINE

Bảng 1.2: Ý nghĩa các trường dữ liệu giao dịch

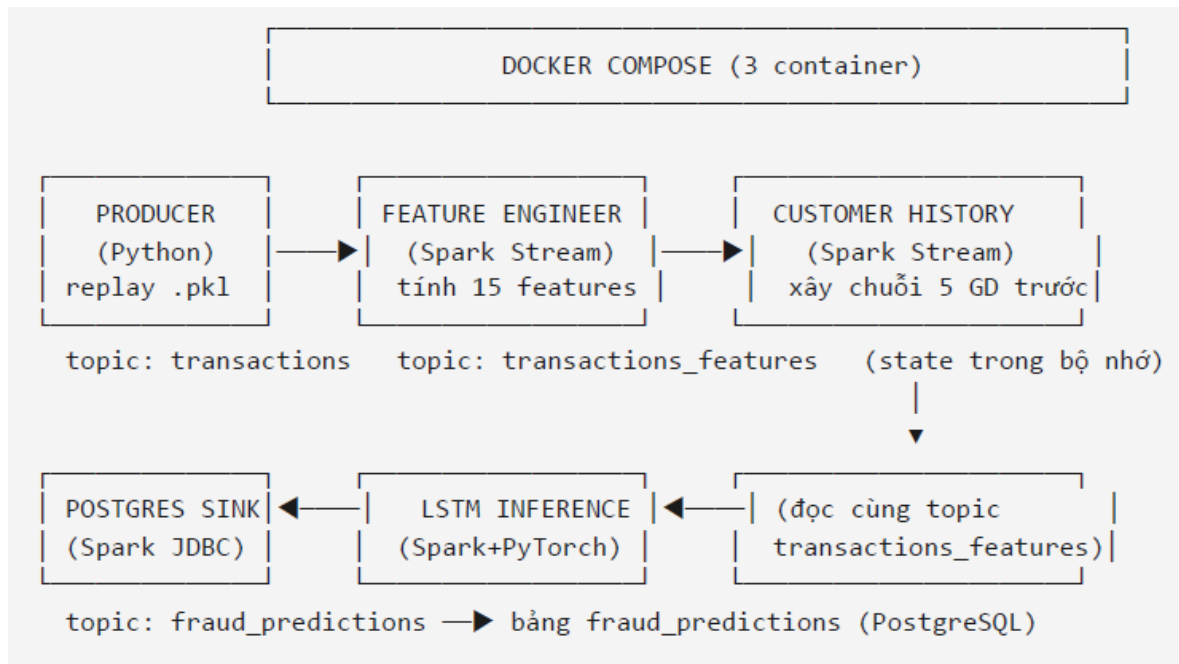
Cột	Ý nghĩa
TRANSACTION_ID	Mã định danh duy nhất của giao dịch (khóa chính duy nhất).
TX_DATETIME	Thời điểm diễn ra giao dịch.
CUSTOMER_ID	Mã định danh của khách hàng. Mỗi khách hàng có một mã định danh duy nhất.
TERMINAL_ID	Mã định danh của đơn vị chấp nhận thanh toán (hay chính xác hơn là của thiết bị đầu cuối). Mỗi thiết bị đầu cuối có một mã định danh duy nhất.
TX_AMOUNT	Số tiền giao dịch.
TX_TIME_SECONDS	Thời gian tính theo giây.
TX_TIME_DAYS	Số ngày kể từ thời điểm giao dịch đầu tiên diễn ra.
TX_FRAUD	Một biến nhị phân, với giá trị 0 cho giao dịch hợp lệ hoặc giá trị 1 cho giao dịch gian lận.

1.4 Sơ Đồ Tổng Thể Pipeline

DOCKER COMPOSE (3 container)



1.4. SƠ ĐỒ TỔNG THỂ PIPELINE



Hình 1.1: Quá trình real time.

Pipeline gồm 6 giai đoạn chạy song song trong các cửa sổ terminal riêng biệt (do `start_realtime.bat` khởi động):

1. Kafka Producer: phát lại giao dịch lịch sử vào Kafka.
2. Feature Engineering: tính 15 đặc trưng cho từng giao dịch.
3. Customer History: xây chuỗi 5 giao dịch trước của mỗi khách hàng (đầu vào chuỗi cho LSTM).
4. LSTM Inference: dự đoán xác suất gian lận bằng mô hình LSTM + Attention.
5. PostgreSQL Sink: ghi kết quả dự đoán xuống PostgreSQL.

1.5. CÁC KAFKA TOPIC

1.5 Các Kafka Topic

Bảng 1.3: Các kafka topic trong hệ thống

Topic	Giai đoạn ghi	Giai đoạn đọc
transactions	Producer	Feature Engineering
transactions_features	Feature Engineering	Customer History, LSTM Inference
fraud_predictions	LSTM Inference	PostgreSQL Sink

Chương 2

CHI TIẾT QUY TRÌNH

2.1 Giai Đoạn Kafka Producer

Vai trò: mô phỏng nguồn giao dịch thời gian thực bằng cách phát lại dữ liệu lịch sử vào Kafka.

Luồng xử lý chi tiết:

1. Đọc danh sách file .pkl trong thư mục data theo ngày từ 2018-10-01 đến 2020-05-22.
2. Tải và chuẩn hóa từng file ngày: chỉ giữ 8 cột bắt buộc, sắp xếp theo TX_DATETIME, TRANSACTION_ID để đảm bảo thứ tự giao dịch đúng như dữ liệu gốc.
3. Điều khiển tốc độ phát (rate control) dữ liệu.
4. Tạo thông điệp Kafka: giữ nguyên 8 cột gốc, thêm PRODUCER_TIMESTAMP là thời gian sắp gửi giao dịch. Đây chính là mốc thời gian để tính độ trễ end-to-end ở hạ nguồn. TX_FRAUD chỉ là ground truth không được dùng để tính đặc trưng hay dự đoán ở bất kỳ giai đoạn nào.

2.2. GIAI ĐOẠN FEATURE ENGINEERING

5. Gửi tới Kafka: bootstrap server localhost:9092, topic transactions, partition 0.
6. Hỗ trợ nhiều chế độ chạy qua tham số dòng lệnh: `-rate` (10/50/100/500), `-limit` (số giao dịch tối đa, ví dụ 500000), `-print-every`, `-dry-run`, `-data-dir`, `-topic`, `-start-date`, `-end-date`. Nếu chạy hết toàn bộ dữ liệu mà chưa đạt `-limit`, producer tự động lặp lại từ đầu để tiếp tục phát.
7. Kết thúc: khi đạt `-limit`.

2.2 Giai Đoạn Feature Engineering

Vai trò: đọc giao dịch thô từ topic transactions, tính 15 đặc trưng và ghi vào topic transactions_features.

Cách đọc dữ liệu:

- Dùng Spark Structured Streaming (`readStream` với `format("kafka")`, `subscribe = "transactions"`).
- Khi chưa có checkpoint → đọc từ earliest (toàn bộ dữ liệu trong topic); khi có checkpoint → đọc tiếp từ vị trí đã lưu.
- `maxOffsetsPerTrigger = 20000`: giới hạn số bản ghi tối đa mỗi micro-batch để tránh một batch quá lớn bấp nhét vào bộ nhớ Python.
- Giải nén json value theo schema khai báo (`StructType`); bỏ qua các message không parse được.

Cách xử lý trong mỗi micro-batch:

1. Sắp xếp các giao dịch theo TX_DATETIME (giữ nguyên logic lịch sử của notebook huấn luyện).

2.2. GIAI ĐOẠN FEATURE ENGINEERING

2. Với mỗi giao dịch, kiểm tra tính hợp lệ (bỏ qua nếu TRANSACTION_ID, TX_DATETIME, CUSTOMER_ID, TERMINAL_ID bị NULL; mặc định TX_AMOUNT = 0, TX_FRAUD = 0).
3. Tính 15 đặc trưng sau:
- TX_AMOUNT: số tiền giao dịch và danh sách biến đổi đặc trưng dữ liệu dưới đây.

Bảng 2.1: Feature engineering cho dữ liệu giao dịch

Đặc trưng gốc	Kiểu dữ liệu gốc	Phép biến đổi	Số đặc trưng mới	Kiểu đặc trưng mới
TX_DATETIME	Panda	0 nếu giao dịch diễn ra	1	Integer
	Times-tamp	vào ngày trong tuần, 1 nếu giao dịch diễn ra vào cuối tuần. Đặc trưng mới là TX_DURING_WEEKEND.		(0/1)
TX_DATETIME	Panda	0 nếu giao dịch diễn ra	1	Integer
	Times-tamp	trong khoảng từ 06:00 sáng đến 00:00 đêm, 1 nếu giao dịch diễn ra trong khoảng từ 00:00 đêm đến 06:00 sáng. Đặc trưng mới là TX_DURING_NIGHT.		(0/1)

Tiếp tục ở trang sau

2.2. GIAI ĐOẠN FEATURE ENGINEERING

Bảng 2.1 – tiếp theo

Đặc trưng gốc	Kiểu dữ liệu	Phép biến đổi	Số đặc trưng mới	Kiểu đặc trưng mới
CUSTOMER_ID	Categorical	Số lượng giao dịch của khách hàng trong n ngày gần nhất, với $n \in \{1, 7, 30\}$. Các đặc trưng mới có dạng CUSTOMER_ID_NB_TX_nDAY_WINDOW.	3	Integer
CUSTOMER_ID	Categorical	Giá trị chỉ tiêu trung bình của khách hàng trong n ngày gần nhất, với $n \in \{1, 7, 30\}$. Các đặc trưng mới có dạng CUSTOMER_ID_AVG_AMOUNT_nDAY_WINDOW.	3	Real
TERMINAL_ID	Categorical	Số lượng giao dịch trên terminal trong $n + d$ ngày gần nhất, với $n \in \{1, 7, 30\}$ và $d = 7$. Tham số d được gọi là độ trễ và sẽ được thảo luận sau. Các đặc trưng mới được gọi là TERMINAL_ID_NB_TX_nDAY_WINDOW.	3	Integer

Tiếp tục ở trang sau

2.3. GIAI ĐOẠN CUSTOMER HISTORY

Bảng 2.1 – tiếp theo

Đặc trưng gốc	Kiểu dữ liệu	Phép biến đổi	Số đặc trưng mới	Kiểu đặc trưng mới
TERMINAL_ID	Categorical	Số lượng gian lận trung bình trên thiết bị đầu cuối trong $n + d$ ngày gần nhất, với $n \in \{1, 7, 30\}$ và $d = 7$. Tham số d được gọi là độ trễ và sẽ được thảo luận sau. Các đặc trưng mới được gọi là <code>TERMINAL_ID_RISK_nDAY_WINDOW</code> .	3	Real

2.3 Giai Đoạn Customer History

Vai trò: xây dựng chuỗi (sequence) 5 giao dịch gần nhất của từng khách hàng, đúng dạng đầu vào mà mô hình LSTM cần.

Cách hoạt động:

1. Đọc topic `transactions_features` bằng Spark Structured Streaming.
2. Duy trì state `customer_history` (defaultdict + deque) lưu tối đa `SEQ_LEN = 5` giao dịch gần nhất của mỗi `CUSTOMER_ID`.
3. Với mỗi giao dịch hiện tại (gọi là T6): $X =$ vector đặc trưng của 5 giao dịch trước đó $T1 \rightarrow T5$ (mỗi giao dịch là 1 vector 15 đặc trưng) $\rightarrow$ shape (5, 15). $y =$ nhãn `TX_FRAUD` của giao dịch hiện tại. Giao dịch hiện tại KHÔNG được chèn vào X (tránh target/current-row leakage).

2.4. GIAI ĐOẠN LSTM INFERENCE

4. Sau khi xây xong X, giao dịch hiện tại mới được nối vào lịch sử và cắt còn 5 phần tử gần nhất.
5. Kiểm tra cảnh báo nếu toàn bộ giá trị của X bằng 0 (giúp phát hiện lỗi ở Feature Engineering hoặc Kafka).

2.4 Giai Đoạn LSTM Inference

Vai trò: tải mô hình LSTM + Attention đã huấn luyện, dự đoán xác suất gian lận cho từng giao dịch và ghi kết quả vào topic `fraud_predictions`.

Cách hoạt động chi tiết:

1. Đọc dữ liệu: Spark Structured Streaming đọc topic `transactions_features`.
2. Xây chuỗi: duy trì lịch sử 5 giao dịch gần nhất của mỗi khách hàng trong bộ nhớ. Với mỗi giao dịch hiện tại: $X =$ chuỗi 5 giao dịch trước, mỗi giao dịch 15 đặc trưng. Khi khách hàng chưa có đủ 5 giao dịch, chuỗi được left zero-padding (chèn các vector 0 vào đầu) để vẫn tạo được dự đoán cho mọi giao dịch:

$$T1 \rightarrow X = [0,0,0,0,0]$$

$$T2 \rightarrow X = [0,0,0,0,T1]$$

$$T3 \rightarrow X = [0,0,0,T1,T2]$$

$$T4 \rightarrow X = [0,0,T1,T2,T3]$$

$$T5 \rightarrow X = [0,T1,T2,T3,T4]$$

$$T6 \rightarrow X = [T1,T2,T3,T4,T5]$$

3. Chuẩn hóa đặc trưng: các đặc trưng real-time là dạng thô (RAW). Mô hình được huấn luyện trên dữ liệu đã chuẩn hóa bằng `StandardScaler` (fit trên cửa sổ huấn luyện 2018-07-11 $\rightarrow$ 2018-07-18). Do đó mỗi giá trị được chuẩn hóa trước khi đưa vào mô hình bằng đúng mean/std của tập huấn luyện.

2.5. GIAI ĐOẠN POSTGRESQL SINK

4. Dự đoán theo batch (BATCHED INFERENCE): gộp tất cả mẫu X của batch thành tensor (N, 5, 15).

Ghi nhận PREDICTION_START_TIMESTAMP (UTC) và `start = perf_counter()`.

Gọi `model(x_batch)` một lần trong `torch.no_grad()`.

Ghi nhận PREDICTION_END_TIMESTAMP và `end = perf_counter()`.

Và `batch_latency_ms = (end - start) * 1000` đây là prediction latency của cả batch (mỗi giao dịch trong batch nhận cùng giá trị này).

5. Tính kết quả: `probability = đầu ra sigmoid của mô hình (float 0..1)`.

`FRAUD_PREDICTION = 1` nếu `probability >= THRESHOLD`, ngược lại 0.

Ghi nhận `END_TO_END_LATENCY_MS` (tạm tính tại đây) = `PREDICTION_END_TIMESTAMP - PRODUCER_TIMESTAMP`. Lưu các thống kê latency (`current batch + cumulative`).

6. Ghi Kafka: tạo DataFrame `output_df` với schema cố định gồm các cột:

`TRANSACTION_ID, TX_DATETIME, PRODUCER_TIMESTAMP, CUSTOMER_ID, TERMINAL_ID, TX_AMOUNT, FRAUD_PROBABILITY, FRAUD_PREDICTION, THRESHOLD, TX_FRAUD,`

`PREDICTION_START_TIMESTAMP, PREDICTION_END_TIMESTAMP, PREDICTION_LATENCY_MS, END_TO_END_LATENCY_MS.`

Ghi JSON vào topic `fraud_predictions`, `key = TRANSACTION_ID`.

2.5 Giai Đoạn PostgreSQL Sink

Vai trò: đọc kết quả dự đoán từ topic `fraud_predictions` và ghi xuống bảng `fraud_predictions` trong PostgreSQL đây là nơi chứa dữ liệu cuối cùng để đánh giá.

2.5. GIAI ĐOẠN POSTGRESQL SINK

Cách hoạt động chi tiết:

1. Spark Structured Streaming đọc topic `fraud_predictions`, parse JSON theo schema.
2. Chuyển đổi các trường timestamp sang kiểu dữ liệu tương ứng.
3. Tạo `SINK_TIMESTAMP = current_timestamp()` tại thời điểm xử lý batch (mốc thời gian "đầu ra cuối cùng" của pipeline).

4. Tính lại `END_TO_END_LATENCY_MS` chính xác tại điểm cuối:

`END_TO_END_LATENCY_MS = unix_micros(SINK_TIMESTAMP) - unix_micros(PRODUCER_TIMESTAMP)`. Dùng `unix_micros()` để giữ độ chính xác đến micro-giây.

Không dùng `TX_DATETIME` (thời điểm giao dịch trong bộ dữ liệu gốc) cho latency real-time.

5. Sắp xếp theo `TX_DATETIME`, gộp về 1 partition (`coalesce(1)`) để các dòng được ghi tuần tự qua một kết nối JDBC giữ nguyên thứ tự trong PostgreSQL.
6. Ghi qua JDBC: URL `jdbc:postgresql://postgres:5432/fraud_detection`, bảng `fraud_predictions`. User/password `fraud/fraud123`, driver `org.postgresql.Driver`, `batchsize=1000`, `stringtype = unspecified`, mode `append`.

2.5. GIAI ĐOẠN POSTGRESQL SINK

Bảng 2.2: Bảng fraud_predictions (schema)

Cột	Kiểu	Ý nghĩa
TRANSACTION_ID	int	Mã giao dịch (khóa chính)
TX_DATETIME	timestamp	Thời điểm giao dịch gốc
PRODUCER_TIMESTAMP	timestamp	Thời điểm producer gửi
CUSTOMER_ID	int	Mã khách hàng
TERMINAL_ID	int	Mã terminal
TX_AMOUNT	double	Số tiền giao dịch
FRAUD_PROBABILITY	double	Xác suất gian lận do mô hình trả ra
FRAUD_PREDICTION	int	Nhãn dự đoán (0/1)
THRESHOLD	double	Ngưỡng được sử dụng để chuyển xác suất thành nhãn dự đoán
TX_FRAUD	int	Nhãn thật (ground truth)
PREDICTION_START_TIMESTAMP	timestamp	Mốc thời gian bắt đầu quá trình inference
PREDICTION_END_TIMESTAMP	timestamp	Mốc thời gian kết thúc quá trình inference
PREDICTION_LATENCY_MS	double	Độ trễ dự đoán, tính bằng mili-giây (ms)
END_TO_END_LATENCY_MS	double	Độ trễ từ thời điểm producer gửi giao dịch đến khi kết quả được ghi vào sink
SINK_TIMESTAMP	timestamp	Thời điểm kết quả được ghi vào PostgreSQL

Chương 3

CHỈ SỐ ĐÁNH GIÁ VÀ KẾT QUẢ

3.1 Đo Lường Hiệu Năng Thời Gian Thực

3.1.1 Các mốc thời gian

Bảng 3.1: Các mốc thời gian trong pipeline real-time

Mốc	Được tạo bởi	Ý nghĩa
TX_DATETIME	Dataset gốc	Thời điểm giao dịch lịch sử (không dùng cho latency)
PRODUCER_TIMESTAMP	Producer	Thời điểm giao dịch thực sự được gửi vào Kafka
PREDICTION_START_TIMESTAMP	LSTM Inference	Thời điểm bắt đầu inference
PREDICTION_END_TIMESTAMP	LSTM Inference	Thời điểm kết thúc inference
SINK_TIMESTAMP	PostgreSQL Sink	Thời điểm giao dịch được ghi xuống PostgreSQL

3.1. ĐO LƯỜNG HIỆU NĂNG THỜI GIAN THỰC

3.1.2 Các chỉ số độ trễ

Prediction latency: thời gian mô hình suy diễn (không gồm Kafka/Spark):

$$\text{PREDICTION_LATENCY_MS} = \text{PREDICTION_END_TIMESTAMP} - \text{PREDICTION_START_TIMESTAMP}$$
$$= (\text{perf_counter_end} - \text{perf_counter_start}) \times 1000$$

End-to-end latency: thời gian trọn vòng từ lúc producer gửi đến lúc ghi vào PostgreSQL (bao gồm Kafka, Spark, Feature Engineering, LSTM, Sink):

$$\text{END_TO_END_LATENCY_MS} = \text{unix_micros}(\text{SINK_TIMESTAMP}) - \text{unix_micros}(\text{PRODUCER_TIMESTAMP})$$

3.1.3 Throughput

Throughput (tx/s) = số giao dịch đã xử lý / thời gian thực tế của pipeline

Trong `evaluate_realtime.py`, throughput được đo từ khoảng thời gian thực (measurement start → measurement end), không dùng `TX_DATETIME`.

3.1.4 Bộ công cụ đánh giá

Sau khi pipeline chạy xong, kết nối PostgreSQL, đọc toàn bộ bảng `fraud_predictions` và tính:

- Dataset summary: tổng giao dịch, số gian lận thật/giả.
- Confusion matrix: TN, FP, FN, TP.
- Classification metrics: Accuracy, Precision, Recall, F1, FPR.
- Probability metrics: ROC-AUC, Average Precision.
- Latency statistics: trung bình, P50, P95, P99, min, max của prediction latency và end-to-end latency.
- Throughput: giao dịch/giây.

3.2. KẾT QUẢ THỰC TẾ

- Precision@Top-K: Precision@50, @100, @200 theo xác suất dự đoán giảm dần.

3.2 Kết Quả Thực Tế

Mỗi benchmark xử lý 500.000 giao dịch với một tốc độ phát khác nhau, pipeline được reset hoàn toàn trước mỗi vòng.

3.2.1 Bảng tổng hợp so sánh 3 tốc độ

Bảng 3.2: Thống kê giao dịch và kết quả dự đoán theo tốc độ xử lý

Chỉ số	50 tx/s	100 tx/s	500 tx/s
Tổng giao dịch	500.000	500.000	500.000
Gian lận thật (ground truth)	3.592	3.592	3.592
Dự đoán gian lận	2.429	2.429	2.425

Bảng 3.3: Ma trận nhầm lẫn theo tốc độ xử lý

Chỉ số	50 tx/s	100 tx/s	500 tx/s
TN	496.252	496.252	496.252
FP	156	156	156
FN	1.319	1.319	1.323
TP	2.273	2.273	2.269

Bảng 3.4: Các chỉ số đánh giá mô hình theo tốc độ xử lý

Chỉ số	50 tx/s	100 tx/s	500 tx/s
Accuracy	0,99705	0,99705	0,99704
Precision	0,93578	0,93578	0,93567
Recall	0,63280	0,63280	0,63168
F1-score	0,75502	0,75502	0,75420
FPR	0,00031	0,00031	0,00031
ROC-AUC	0,97205	0,97205	0,97199
Average Precision	0,80547	0,80545	0,80573
Precision@50 / @100 / @200	1,0	1,0	1,0

3.2. KẾT QUẢ THỰC TẾ

Bảng 3.5: Độ trễ dự đoán (prediction latency)

Chỉ số	50 tx/s	100 tx/s	500 tx/s
Trung bình (ms)	64,02	137,75	5.329,14
P50 (ms)	27	53	4.671
P95 (ms)	205	396	9.884
P99 (ms)	914	2.583	9.884
Min (ms)	3	10	142
Max (ms)	1.099	2.583	9.884

Bảng 3.6: Độ trễ end-to-end (từ Producer $\rightarrow$ PostgreSQL Sink)

Chỉ số	50 tx/s	100 tx/s	500 tx/s
Trung bình (ms)	16.373	22.058	416.443
P50 (ms)	14.755	19.332	372.603
P95 (ms)	23.385	30.320	625.584
P99 (ms)	54.535	110.375	649.157
Max (ms)	137.565	140.489	659.551

Bảng 3.7: Throughput thực tế

Chỉ số	50 tx/s	100 tx/s	500 tx/s
Số giao dịch	500.000	500.000	500.000
Thời lượng pipeline (giây)	10.012,8	5.014,4	1.619,9
Throughput thực (tx/s)	49,94	99,71	308,66

3.2.2 Phân tích kết quả

1. Chất lượng dự đoán không đổi theo tốc độ: các chỉ số Accuracy/Precision/Recall/F1/R_{0.5}AUC/Average Precision/Precision@50, @100, @200 gần như giống hệt nhau ở cả 3 mức tốc độ. Điều này cho thấy tốc độ phát không ảnh hưởng đến độ chính xác của mô hình. Độ trễ chỉ ảnh hưởng đến thời điểm có kết quả, không ảnh hưởng đến nội dung dự đoán.
2. Độ chính xác tổng thể rất cao (Accuracy $\approx$ 99,7%) với Precision $\approx$ 93,6% nghĩa là khi hệ thống báo gian lận $\sim$ 93,6% trường hợp là đúng. Recall $\approx$ 63,2% cho thấy hệ thống bắt được khoảng 2/3 số vụ gian lận thật, mức

3.3. MỨC TIÊU THỤ TÀI NGUYÊN HỆ THỐNG

hợp lý cho bài toán phát hiện gian lận trong thời gian thực với ngưỡng cố định.

3. Latency tăng nhanh theo tốc độ:

Ở 50 tx/s : prediction latency trung bình chỉ ~ 64 ms, end-to-end trung bình ~ 16 giây, hoàn toàn trong vùng chấp nhận được.

Ở 100 tx/s: prediction latency ~ 138 ms, end-to-end ~ 22 giây, vẫn ổn.

Ở 500 tx/s: prediction latency vọt lên $\sim 5,3$ giây, end-to-end trung bình ~ 416 giây (~ 7 phút), pipeline không theo kịp tốc độ phát; dữ liệu bị ùn ứ trong Kafka.

4. Throughput thực tế ở 500 tx/s chỉ đạt $\sim 308,66$ tx/s (thấp hơn mục tiêu 500 tx/s $\sim 38\%$). Đây chính là nguyên nhân độ trễ tăng vọt: hệ thống xử lý chậm hơn tốc độ dữ liệu đến nên hàng đợi Kafka dài dần.

5. Precision@Top-K = 1,0 ở mọi tốc độ: khi sắp xếp 500.000 giao dịch theo xác suất gian lận giảm dần, toàn bộ 200 giao dịch đứng đầu đều là gian lận thật, điểm rất mạnh nếu muốn sử dụng ngưỡng động (dynamic threshold) hoặc xếp hạng rủi ro.

3.3 Mức tiêu thụ tài nguyên hệ thống

Dữ liệu được ghi lại bởi monitor_resources.py (file resource_metrics.csv). Ví dụ một mẫu đo trong quá trình chạy:

Bảng 3.8: Mức sử dụng tài nguyên của các container

Container	CPU	Memory
spark	$\sim 383\%$	$\sim 1,34$ GiB / 7,68 GiB (17,4%)
kafka	$\sim 7\% - 35\%$	$\sim 1,13$ GiB / 7,68 GiB (14,7%)
postgres	$\sim 0\%$	~ 46 MiB / 7,68 GiB (0,6%)

Nhận xét:

3.4. KẾT LUẬN

- Spark là thành phần tốn tài nguyên nhất (hàng trăm % CPU do chạy 4 streaming job đồng thời, bao gồm cả suy diễn PyTorch) đây là điểm nghẽn chính khi tăng tốc độ giao dịch.
- Kafka tiêu thụ vừa phải ($\sim 1,1$ GiB RAM, CPU dao động theo lưu lượng).
- PostgreSQL rất nhẹ ở mức lưu lượng này.

3.4 Kết Luận

3.4.1 Tóm tắt quá trình real-time

1. Producer phát lại 500.000 giao dịch lịch sử vào Kafka theo tốc độ cấu hình (50/100/500 tx/s), gán PRODUCER_TIMESTAMP làm mốc thời gian thực.
2. Feature Engineering tính 15 đặc trưng (thời điểm, số tiền giao dịch, cửa sổ khách hàng/terminal 1-7-30 ngày).
3. Customer History và LSTM Inference xây chuỗi 5 giao dịch trước của từng khách hàng, chuẩn hóa theo scaler huấn luyện và dự đoán xác suất gian lận bằng mô hình LSTM + Attention.
4. PostgreSQL Sink ghi kết quả kèm SINK_TIMESTAMP, cho phép tính độ trễ end-to-end chính xác.
5. Bộ đánh giá so sánh FRAUD_PREDICTION với ground truth TX_FRAUD và đo latency/throughput.

3.4.2 Kết quả đạt được

- Chất lượng dự đoán ổn định: Accuracy $\approx 99,7\%$, Precision $\approx 93,6\%$, ROC-AUC $\approx 0,972$, Average Precision $\approx 0,8$, Precision@50-100-200 = 1 ở mọi tốc độ.

3.4. KẾT LUẬN

- Độ trễ thấp ở tốc độ thấp: end-to-end ~ 16 giây (50 tx/s), ~ 22 giây (100 tx/s).
- Hệ thống có năng lực xử lý ~ 300 tx/s: ở 500 tx/s pipeline bắt đầu quá tải (end-to-end ~ 7 phút, throughput thực chỉ ~ 309 tx/s).

3.4.3 Hạn chế và hướng cải tiến

1. Nghẽn cổ chai ở 500 tx/s: cần tối ưu Feature Engineering (giảm overhead Python trong foreachBatch), tăng số partition Kafka để song song hóa, hoặc tăng tài nguyên cho container Spark.
2. Trạng thái trong bộ nhớ driver: defaultdict/deque không được checkpoint, khi job khởi động lại giữa chừng, lịch sử sẽ được dựng lại từ đầu. Có thể dùng Spark state store (mapGroupsWithState) để lưu trạng thái bền vững.
3. Ngưỡng cố định: các chỉ số hiệu suất mô hình có thể được cải thiện bằng ngưỡng động hoặc tối ưu threshold.
4. Prediction latency gắn với batch: mọi giao dịch trong một micro-batch nhận cùng latency của batch, có thể giảm bằng cách giảm kích thước batch hoặc dùng streaming inference.